

EXECUTIVE SUMMARY

Target: http://combined-test.com
Scan ID: TORTURE-P5-657b5747
Scan Date: 2026-03-04 18:24:38
Scan Duration: N/A
Total Requests Sent: 28
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 4 vulnerabilities detected

Severity Breakdown: Critical: 1 | High: 2 | Medium: 1 | Low: 0
AI Inference Calls: 0
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- Overall Risk Level: High due to multiple vulnerabilities across all categories

EXECUTIVE STRATEGIC IMPACT

```
{
  "Strategic_Attack_Path_Analysis": [
    "An attacker could begin by exploiting the SQL_INJECTION vulnerability in the application's backend database, allowing them to gain unauthorized access and manipulate or exfiltrate sensitive data.",
    "Following this, they would leverage the CSRF (Cross-Site Request Forgery) weakness to trick authenticated users into performing actions on behalf of themselves without their knowledge, potentially leading to further privilege escalation or lateral movement within the system.",
    "Finally, by exploiting the PATH_TRAVERSAL vulnerability in a file upload mechanism, an attacker could inject malicious payloads that execute arbitrary code with elevated privileges, enabling them to achieve full system compromise and maintain persistent access."
  ]
}
```

DETAILED FINDINGS

FILTER: INJECTION & FUZZING

Finding #1: SQL Injection

CRITICAL

CWE: CWE-89

CVSS Score: 10.0 (CRITICAL)

THREAT SCORE: 100/100

Description:

- The endpoint /api/v1/endpoint_0 is vulnerable to SQL injection attacks.
- User input in the request payload can be manipulated to inject malicious SQL code.
- This vulnerability could allow unauthorized data access or manipulation.

Impact:

- Potential exposure of sensitive user information (e.g., credentials, personal data).
- Risk of data tampering and integrity compromise.
- Possibility of privilege escalation leading to full system compromise.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SQL_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_0

URL: http://test-target-torture-.com/api/v1/endpoint_0

Analysis: The application failed to properly sanitize user input before constructing SQL queries.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: SQL Injection
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement parameterized queries or prepared statements in the backend code.
- Use input validation and sanitization for all user-supplied data before processing.
- Apply least privilege principles: restrict database permissions to only necessary operations.

Recommended Code Fix:

```
/* Example of a secure query using parameter binding */  
SELECT * FROM users WHERE username = ? AND password = ?;
```

FILTER: [ANSWER] AUTHENTICATION GATES

Finding #2: Cross-Site Request Forgery

HIGH

CWE: CWE-352
CVSS Score: 7.5 (HIGH)

THREAT SCORE: 75/100

Description:

- The target endpoint is susceptible to Cross-Site Request Forgery attacks.
- Attackers could trick authenticated users into performing unintended actions on the vulnerable service.
- Proper CSRF protection mechanisms are required to mitigate this risk.

Impact:

- Unauthorized actions may be performed by malicious actors, leading to data manipulation or unauthorized changes.
- User trust and confidence in the affected system can be compromised due to potential security breaches.
- The vulnerability could result in financial losses, reputational damage, and legal liabilities for the organization.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The application failed to validate and sanitize user input, allowing malicious requests to be processed.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	Direct reference to target resource.

Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category:	Cross-Site Request Forgery
Raw Payload:	test_payload_1
Encoded:	746573745f7061796c6f61645f31
Encoding Type:	None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json
{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Implement CSRF tokens on all request forms and ensure they are validated server-side before processing requests.
- - Use anti-CSRF headers (e.g., X-XSRF-TOKEN) to protect against forged requests from malicious websites.
- - Employ secure coding practices, such as validating user input thoroughly and using HTTPOnly cookies for token storage.

Recommended Code Fix:

```
// Example of a CSRF protection mechanism in Node.js with Express
const express = require('express');
const app = express();
app.use(express.json());

// Generate a unique CSRF token on each request
function generateCsrfToken() {
  return Math.random().toString(36).substring(2, 15) + Math.random().toString(36)
}

// Middleware to validate the CSRF token in requests
```

```
app.use((req, res, next) => {  
  const csrfToken = req.headers['x-xsrf-token'] || generateCsrfToken();  
  if (csrfToken !== req.body.csrf_token) {  
    return res.status(403).send('Invalid or missing CSRF token');  
  }  
  next();  
});
```

FILTER: OBJECT REFERENCES (IDOR)

Finding #3: Path Traversal

CRITICAL

CWE: CWE-22
CVSS Score: 9.5 (CRITICAL)

THREAT SCORE: 95/100

Description:

- Vulnerability 'Path Traversal' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PATH_TRAVERSAL' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_2
URL: http://test-target-torture-.com/api/v1/endpoint_2
Analysis: The application failed to properly validate and sanitize user input, allowing arbitrary path traversal.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_2	test_payload_2	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Path Traversal
Raw Payload: test_payload_2
Encoded: 746573745f7061796c6f61645f32
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_2'
```

HTTP Traffic Snapshot:

Request:

```
POST http://test-target-torture-.com/api/v1/endpoint_2 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement strict input validation.

Recommended Code Fix:

```
import os
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']

    if not file.filename:
        return jsonify({'error': 'No filename provided'}), 400

    # Validate and sanitize the uploaded path
    sanitized_path = os.path.abspath(os.path.join(app.root_path, 'uploads'))
    absolute_path = os.path.join(sanitized_path, file.filename)

    try:
        with open(absolute_path, 'wb') as f:
            f.write(file.read())
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```



```
return jsonify({'message': 'File uploaded successfully'}), 200
```

FILTER: AUTHENTICATION GATES

Finding #4: Broken Authentication

CRITICAL

CWE: CWE-287
CVSS Score: 10.0 (CRITICAL)

THREAT SCORE: 100/100

Description:

- The endpoint /api/v1/endpoint_3 does not validate authentication credentials.
- User input is directly used in the request without proper authorization checks.
- No rate limiting or IP blocking mechanisms are enforced for this endpoint.

Impact:

- Unauthorized access to sensitive resources and data.
- Potential privilege escalation attacks.
- Data breaches if user information is exposed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'BROKEN_AUTH' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_3
URL: http://test-target-torture-.com/api/v1/endpoint_3
Analysis: The application failed to validate and sanitize user input, allowing arbitrary payload injection.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_3	test_payload_3	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Broken Authentication

Raw Payload: test_payload_3

Encoded: 746573745f7061796c6f61645f33

Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_3'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_3 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement proper authentication validation (e.g., JWT, OAuth) on the server-side.
- Add input sanitization and parameterized queries to prevent injection attacks.
- Enforce rate limiting and IP blocking for this endpoint to mitigate brute force attempts.

Recommended Code Fix:

```
// Example secure code snippet in Python
import jwt

def validate_auth_token(token):\n    try:\n        payload = jwt.decode(token, S
```

SCAN TIMELINE

[Orchestrator] TARGET_ACQUIRED - 2026-03-04 18:24:38

[Sigma] JOB_ASSIGNED - 2026-03-04 18:24:38

[Gamma] LOG - 2026-03-04 18:24:38

[Gamma] LOG - 2026-03-04 18:24:38

[Kappa] LOG - 2026-03-04 18:24:38

[Kappa] LOG - 2026-03-04 18:24:38

[Alpha] LOG - 2026-03-04 18:24:38

[Alpha] LOG - 2026-03-04 18:24:39

[Alpha] LOG - 2026-03-04 18:24:39

[Alpha] LOG - 2026-03-04 18:24:39

[Beta] LOG - 2026-03-04 18:24:39

[Gamma] LOG - 2026-03-04 18:24:39

[Kappa] LOG - 2026-03-04 18:24:39

[Gamma] LOG - 2026-03-04 18:24:39

[Gamma] LOG - 2026-03-04 18:24:39

[Kappa] LOG - 2026-03-04 18:24:39

[Beta] LOG - 2026-03-04 18:24:39

[Alpha] LOG - 2026-03-04 18:24:40

[Gamma] LOG - 2026-03-04 18:24:40

[Kappa] LOG - 2026-03-04 18:24:40

[Alpha] LOG - 2026-03-04 18:24:40

[Kappa] LOG - 2026-03-04 18:24:40

[Gamma] VULN_CONFIRMED - 2026-03-04 18:24:40

[Gamma] VULN_CONFIRMED - 2026-03-04 18:24:40

[Gamma] VULN_CONFIRMED - 2026-03-04 18:24:41

[Gamma] VULN_CONFIRMED - 2026-03-04 18:24:41

[Gamma] VULN_CONFIRMED - 2026-03-04 18:24:41

[Kappa] GI5_LOG - 2026-03-04 18:24:48